RV College of Engineering®

Go, change the world®

MACHINE LEARNING OPERATIONS

Case Study – MedInsure Health Claim Approval Model

Team Members

USN	Name
1RV23AI014	Amudhan S
1RV23AI029	Dhaksha Muthukumaran
1RV23AI032	Dhruv Patankar
1RV23AI001	Aaditey Chalva

PROBLEM DESCRIPTION

MedInsure, a health-insurance company, has developed a machine-learning model to automatically score and approve low-risk claims under ₹25,000. The model was trained using five years of historical claim records, hospital types, diagnosis codes, treatment cost ranges, customer risk profiles, and fraud-flag indicators. During development, the model achieved strong accuracy and passed internal validation. Now, the company plans to move the model to the production environment.

Issues Encountered During Pre-Production Testing:

1. Runtime Environment Mismatch
2. Data Access; Schema Changes
3. Model Risk Evaluation Concerns
4. Quality Assurance; Stress Testing
5. Reproducibility Issues
6. Security; Adversarial Inputs

QUESTION

Explain why stress testing is necessary for ML models in production. Suggest at least two methods to make the API stable under heavy claim-submission load.

WHY STRESS TESTING

1. **Non-linear resource behavior**: Even a well-performing model can crash when traffic spikes.
2. **Real-world traffic is unpredictable**: A model that works fine in staging will fail in production (claims - burst).
3. **Customer experience and business impact**: Slow or failed claims frustrate customers, damage trust, and may lead to regulatory complaints in health insurance.
4. **SLA compliance and cost control**: Stress testing proves whether you meet SLAs and helps avoid over-provisioning expensive infrastructure.
5. **Hidden bottlenecks surface only under load**: Issues like database connection pooling exhaustion, model batching inefficiencies etc. typically appear only when hundreds/thousands of requests arrive concurrently.

METHODS TO MAKE API STABLE

1. Autoscaling the Inference Service : When thousands of claim requests arrive simultaneously, fixed compute capacity will cause timeouts, ejected requests, model slowdowns

2. Batch Processing + Asynchronous Queues:

- Task queues (RabbitMQ, Kafka, SQS),
- Batch inference (combine multiple claim requests at once)

This prevents real-time overload and ensures the API doesn't crash when burst loads happen.

3. Caching Frequent Predictions or Static Data

- Recent ML outputs,
- Static preprocessed features,

Using Redis or CDN-level caching reduces repeated processing and keeps latency predictable.

4. Rate Limiting + Throttling

- Control how many requests per second one client sends
- Burst traffic from automated claim-processing systems

5. Circuit Breakers + Timeouts

Circuit breakers help by: cutting off requests to unhealthy services, returning fallback responses or queued processing, preventing total system collapse